Ideal Braking Curve

Technical Report

By Inaaya Rahim

Formula Student Society

Submission date: 29 October 2026

Table of Contents

1. Introduction and Objective *3*

2. Programming Language Choice *3*

3. Theory and Equations *4*

3.1. Vehicle Parameters 4

3.2. Longitudinal Weight Transfer 4

3.3. Front and Rear Braking Forces 5

3.4. Calculating the Braking Curve 5

4. Assumptions and Modelling Decisions *6*

4.1. Vehicle and Road Condition 6

4.2. Braking Deceleration 6

4.3. Static Weight Distribution 7

4.4. Range of Friction Coefficients 7

5. Code Structure and Implementation *7*

5.1. Libraries and Vehicle Parameters 7

5.2. Calculation Functions 7

5.3. Results and Plotting 8

6. How to Run the Program *9*

7. Limitations and Possible Improvements *10*

7.1. Limitations 10

7.2. Possible improvements 10

8. References *11*

1. Introduction and Objective

This report describes the development of a program that calculates and plots an ideal braking curve for a vehicle. The ideal braking curve shows the relationship between the braking forces at the front and rear axles, as weight shifts towards the front of the vehicle during braking (longitudinal weight transfer). The program performs the calculations in Python, generates a plot and presents the results in a table, making the results accessible to other Formula Student team members.

2. Programming Language Choice

Python was chosen for this project because I am already familiar with the language, allowing me to implement the calculations efficiently. Its concise and readable syntax also makes the code easier for other Formula Student team members to understand, maintain and modify.

The NumPy and Matplotlib libraries were particularly useful for this task. In this program, NumPy supports the numerical calculations across an array of tyre friction coefficients (NumPy Developers, n.d.), allowing calculations to be performed across multiple values. Matplotlib is used to generate the ideal braking curve and the friction coefficient reference lines, add labelled axes and save the resulting plot as a PNG image (Matplotlib Development Team, n.d.).

To improve the program's maintainability and reusability, descriptive variable and function names, comments explaining key sections of the code, and global constants defined at the beginning of the program have been implemented. These constants store the vehicle parameters in SI units, such as gravitational acceleration, vehicle mass, centre-of-gravity height and wheelbase, allowing the values to be easily identified and changed when required. The calculations are separated into functions, making individual operations easier to maintain and reuse. This also means that the program can be adapted for further analysis by simply changing the vehicle parameters or the range of tyre friction coefficients, without needing to recalculate the results manually.

3. Theory and Equations

3.1. Vehicle Parameters

The ideal braking curve is calculated using the supplied equations, which account for longitudinal weight transfer and the braking forces at the front and rear axles. These calculations show how the braking force is distributed between the two axles as the tyre-road friction coefficient changes.

The vehicle parameters listed in Table 1 are defined as global constants at the beginning of the program. This makes them easy to locate and modify if different vehicle specifications need to be used.

Table 1 - Vehicle parameters used in the braking curve calculations

Parameter	Symbol	Value	Unit
Gravitational acceleration	g	9.81	m/s^2
Vehicle mass	m	280	kg
Centre of gravity height	h_{cg}	0.25	m
Wheelbase	d_{wheel}	1.590	m
Front weight distribution	ψ_{front}	0.5	-

The front weight distribution value of 0.5 means that 50% of the vehicle's static weight is supported by the front axle, while the remaining 50% is supported by the rear axle.

The symbols used in the supplied equations are defined in Table 2, together with their meanings and units.

Table 2 - Symbols used in the equations

Symbol	Meaning	Unit
ΔW	Longitudinal weight transfer during braking	N
a	Magnitude of braking deceleration	m/s^2
$F_{slip,front}$	Front axle braking force at the tyre-slip limit	N
$F_{slip,rear}$	Rear axle braking force at the tyre-slip limit	N
μ_{tyre}	Tyre-road friction coefficient	-

3.2. Longitudinal Weight Transfer

When a vehicle brakes, its load shifts towards the front axle, increasing the load on the front wheels and decreasing the load on the rear wheels (Suspensions Explained, 2022).

Longitudinal weight transfer equation:

$$\Delta W = \frac{amh_{cg}}{d_{wheel}}$$

The amount of load transferred depends on the vehicle's mass, braking deceleration, centre of gravity height and wheelbase. A higher centre of gravity or greater deceleration increases weight transfer, while a longer wheelbase reduces it.

The program uses NumPy to generate an array of tyre-road friction coefficients and calculates the corresponding braking deceleration weight transfer for each value.

3.3. Front and Rear Braking Forces

Front axle:

$$F_{slip,front} = \mu_{tyre}(mg\psi_{front} + \Delta W_{front})$$

The front axle's braking force depends on its initial load and the additional load transferred to it during braking.

Rear axle:

$$F_{slip,rear} = \mu_{tyre}(mg(1 - \psi_{front}) - \Delta W_{front})$$

The rear axle's braking force depends on its initial load minus the load transferred away from it during braking.

The tyre-road friction coefficient determines the maximum braking force that can be generated at each axle before the tyres reach their slip limit (Suspensions Explained, 2022).

In the program, the weight transfer is calculated first, followed by the front and rear braking forces for each tyre-road friction coefficient. NumPy arrays are used to store the calculated forces, which are then used to generate the ideal braking curve.

3.4. Calculating the Braking Curve

The program uses NumPy's `linspace()` function to generate 11 tyre-road friction coefficient values ranging from 0.0 to 1.0 inclusive in increments of 0.1.

For each coefficient, the program calculates the corresponding front and rear braking forces.

Matplotlib is then used to plot the front braking force on the horizontal axis and the rear braking force on the vertical axis, producing the ideal braking curve. The plot also includes diagonal reference lines representing different tyre-road coefficients. This makes it easier to compare the braking forces at different levels of available tyre grip.

4. Assumptions and Modelling Decisions

4.1. Vehicle and Road Condition

The model assumes that the vehicle brakes on a flat, level road, with other external longitudinal forces like aerodynamic drag, ignored. The same tyre-road friction coefficient is assumed for both the front and rear axles, and the vehicle's mass and dimensions are treated as constant throughout the calculations.

The tyre-road friction coefficient is varied from 0.0 to 1.0 in increments of 0.1, following a similar range used in the reference video (Suspensions Explained, 2022). This allows the braking forces to be compared using different levels of tyre-road friction.

4.2. Braking Deceleration

The program assumes that the braking deceleration is equal to the tyre-road friction coefficient multiplied by gravitational acceleration:

$$a = \mu_{tyre} g$$

This assumes that the vehicle can only brake as much as the tyre-road friction allows. As the friction coefficient increases, the maximum braking deceleration also increases. The calculated deceleration is then used in the longitudinal weight transfer equation to determine how much load shifts from the rear axle to the front axle.

The actual braking deceleration may be different depending on other factors such as the actual braking system.

4.3. Static Weight Distribution

As specified in the assignment, the front weight distribution is set to 0.5. This means that, before braking, 50% of the vehicle's weight is supported by the front axle and the remaining 50% by the rear axle.

This value is used to calculate the initial weight on each axle. During braking, weight transfers from the rear axle to the front axle, changing the weight supported by each axle (Suspensions Explained, 2022).

4.4. Range of Friction Coefficients

The tyre-road friction coefficient is ranged from 0.0 to 1.0 in increments of 0.1. Using steps of 0.1 allows the results to be compared at regular intervals and makes it easier to see how the braking forces change as the friction coefficient increase.

5. Code Structure and Implementation

The program is organised into separate sections to make the code easier to read, understand and modify. It starts by importing the required libraries and declaring vehicle parameters as constants. Then the calculations are carried out using separate functions. After that the results are printed as a table and displayed in a graph.

5.1. Libraries and Vehicle Parameters

NumPy is used for numerical calculations and Matplotlib to create the graph. The vehicle parameters are defined at the beginning of the code as constants, including `GRAVITY_MS2`, `MASS_KG`, `CG_HEIGHT_M`, `WHEELBASE_M` and `FRONT_WEIGHT_DISTRIBUTION`. Keeping these values together makes them easy to find and change if different vehicle specifications need to be used.

5.2. Calculation Functions

The program uses separate functions for each calculation. `calculate_weight_transfer_N()` calculates the weight transferred from the rear axle to the front axle during braking. The functions `calculate_front_braking_force_N()` and `calculate_rear_braking_force_N()` then calculate the braking forces at each axle. The `calculate_braking_curve()` function brings these calculations together and iterates them over each friction coefficient in the chosen range.

Keeping the calculations separate makes the code easier to follow and means changes can be made to one part without having to rewrite the whole program.

5.3. Results and Plotting

Once the calculations are finished, the program displays the front and rear braking forces in a table, as shown in Figure 1. It then creates a graph showing the front braking force on the horizontal axis and the rear braking force on the vertical axis, shown in Figure 2. I also added diagonal lines to show different friction coefficient values, making it easier to compare the braking forces.

I added a check to make sure that the front and rear braking forces add up to the expected total braking force for each friction coefficient. If the values do not match, the program raises an error. The graph is saved as a PNG image.

Ideal Braking Curve Results

Tyre Friction Coefficient	Front Force (N)	Rear Force (N)
0.0	0.00	0.00
0.1	141.66	133.02
0.2	291.96	257.40
0.3	450.89	373.15
0.4	618.46	480.26
0.5	794.67	578.73
0.6	979.52	668.56
0.7	1173.00	749.76
0.8	1375.13	822.31
0.9	1585.89	886.23
1.0	1805.29	941.51

Figure 1 - Braking force results table

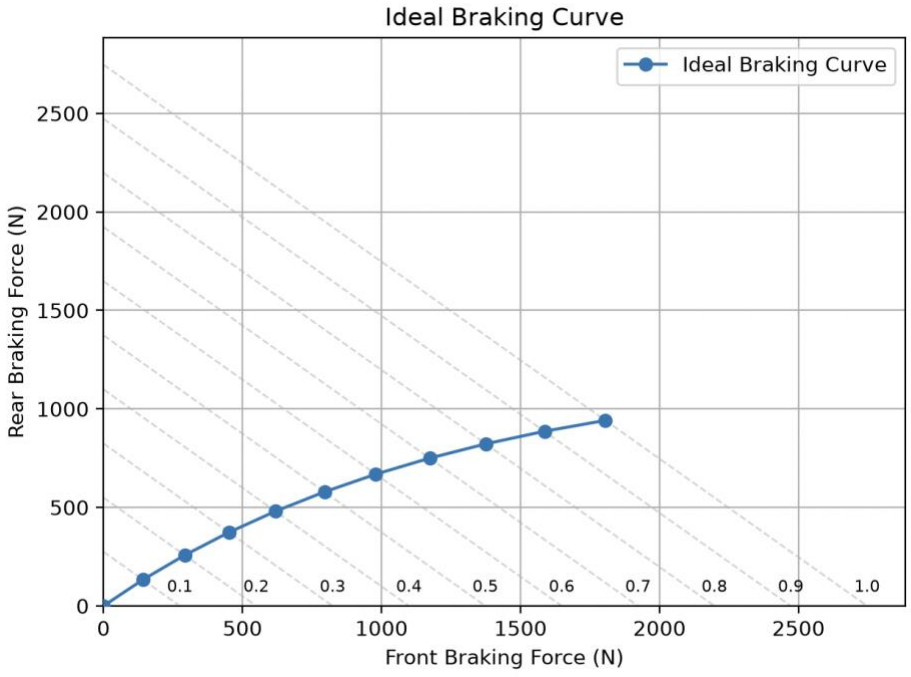


Figure 2 - Ideal braking curve

6. How to Run the Program

The program is written in Python and requires NumPy and Matplotlib to run. Follow the steps below to set up and run the program.

Step 1: Install Python

Install Python 3 from the official website: https://www.python.org/downloads/

Step 2: Download the Project Files

Download or clone the project repository from GitHub:

https://github.com/InaayaRahim/ideal-braking-curve

All the project files must be kept in the same folder.

Step 3: Open the Project Folder

Open the project folder in a code editor, such as Visual Studio Code, if preferred.

Step 4: Install the Required Libraries

The program uses NumPy and Matplotlib, which are listed in the requirements.txt file. A virtual environment is used to keep the libraries for this project separate from other Python projects on the computer.

Open a terminal in the project folder and follow the instructions for your operating system.

For macOS:

Create a virtual environment:

```
python3 -m venv .venv
```

Activate it:

```
source .venv/bin/activate
```

For Windows:

Create a virtual environment:

```
py -m venv .venv
```

Activate it in Command Prompt:

```
.venv\Scripts\activate.bat
```

Alternatively, if using PowerShell:

```
.venv\Scripts\Activate.ps1
```

For both operating systems:

Once the virtual environment is activated, install the required libraries:

```
python -m pip install -r requirements.txt
```

If the activation is successful, the virtual environment's name, .venv, will usually appear at the beginning of the terminal prompt.

Step 5: Run the Program

With the virtual environment activated, run:

```
python main.py
```

The program will calculate the braking forces, print the results in the terminal and display the ideal braking curve. The graph is also saved as a PNG image in the `plots` folder.

Step 6: View the Results

Read the calculated front and rear braking forces in the terminal output. The generated graph can be viewed in the `plots` folder.

7. Limitations and Possible Improvements

7.1. Limitations

The program uses a simplified model of vehicle braking. It assumes that the same tyre-road coefficient applies to both axles and that the braking deceleration is equal to the friction coefficient multiplied by gravitational acceleration. In real situations, braking can also be affected by factors such as road conditions or the braking system.

7.2. Possible improvements

The program could be improved by allowing users to enter different vehicle parameters without having to edit the source code. This would make it easier for team members to use the program with different vehicle specifications and braking conditions.

8. References

Matplotlib Development Team (n.d.) *Matplotlib documentation*. Available at: https://matplotlib.org/stable/index.html (Accessed: 28 October 2026).

NumPy Developers (n.d.) *NumPy documentation*. Available at: https://numpy.org/doc/stable/ (Accessed: 28 October 2026).

Suspensions Explained (2022) *The Ideal Brake Curve* [YouTube video]. Available at: https://www.youtube.com/watch?v=iJ7bf20phWc (Accessed: 28 October 2026).